BuoyUp — Source Code Guide

Oct 19, 2026 · @dungtt

1. Repositories

BuoyUp's source code lives in two repositories: the lending platform (Python backend, web frontend, tests) and the FISCO BCOS chain service (Go and Solidity). The live chain service runs at `https://fs.buoyup.app`.

Repository	Link	Contents	Stack
BuoyUp lending platform	github.com/petertran1811/bouyup	Five AI agents, credit algorithm B1–B12, bank syndicate, chain client, REST API `/v4`, six web pages, backend tests, adversarial TestBench, deployment files	Python 3.12, FastAPI, SQLAlchemy, SQLite; plain HTML/CSS/JS
BuoyUp FISCO BCOS chain service (`sgchain`)	github.com/ffleader1/fisco-bcos-verifier	Smart contracts `Roles`, `WithdrawalLedger`, `AuditTrail`; REST service, explorer, per-bank encryption, bank catalogue, conformance tests	Go ⩾1.22, FISCO BCOS v2.11.1, go-sdk v1.1.1, Solidity 0.6.10

Live endpoints of the chain service: API reference `https://fs.buoyup.app/docs/`, public explorer `https://fs.buoyup.app/explorer/`, health `https://fs.buoyup.app/healthz`.

Size: about 21,600 lines of Python in the application package, 10,800 lines of HTML/CSS/JS (excluding vendored libraries), 6,400 lines of Go and 234 lines of Solidity.

2. README

BuoyUp lends to e-commerce sellers through five AI agents, a fixed credit algorithm (B1–B12), several banks funding each loan together, and an audit trail and encrypted payout orders on FISCO BCOS. Agents choose, explain and check; every money figure comes

from the algorithm. One FastAPI process serves both the API (`/v4/...`) and the web UI (`/app/...`), with state in one SQLite file. Node, Redis and Postgres are not needed to run it.

Lending platform layout

Path	Module
`BackEnd/src/app/core`	Algorithm B1–B9, parameter set v4.0, legal rules and corpus (794 articles), RAG, challenger model, flow replay
`BackEnd/src/app/agents`	Agents 1 (verify), 2 (advise), 3 (control), 4a (payout), 4b (monitor), 5 (post-audit); contract, complaints, LLM tool loop
`BackEnd/src/app/banks`	Partner bank eligibility, B12 split, parallel syndicate round
`BackEnd/src/app/blockchain`	Per-case hash chain, remote FISCO client, bank catalogue sync, encrypted withdrawals
`BackEnd/src/app/data_sources`	Six data sources, sample profiles, consent (Ed25519), shop connection
`BackEnd/src/app/autopilot`	Background loop: Agent 4b every 5 min, Agent 5 every 6 h, seller reports
`BackEnd/src/app/api`	REST API `/v4` (about 120 routes)
`BackEnd/src/app/platform`	Config, database tables, security, capabilities, guards, audit log, LLM clients
`BackEnd/src/Reid`	Face authentication (InsightFace buffalo_l: SCRFD + ArcFace)
`BackEnd/src/CCCD`	PRC ID card OCR (RapidOCR on onnxruntime)
`BackEnd/src/tests`	Backend test suite (pytest, 469 tests)
`BackEnd/src/scripts`	Demo reset, profile generators, RAG index build, challenger training, demo checks
`BackEnd/API_config`	`.env.example` , API contract `API_CONTRACT_V4.md` , chain OpenAPI file

Path	Module
`Frontend/giaodien`	Six pages: landing, login, register, seller workspace, officer/admin console, bank portal
`Frontend/src`	JS/CSS per area: shared, landing, seller, admin, bank, face verification
`Frontend/tests`	API contract check (Python) and UI checks (Node)
`TestBench/`	Adversarial test suite: security cases and 50 end-to-end loan dossiers
`deploy/`	Nginx + HTTPS template, database backup script
`Dockerfile`, `docker-compose.yml`, `run-local.sh`	Container build, one-service compose file, run without Docker
`HANDOVER.md`	Full deployment and operations guide

Chain service layout

Path	Module
`contracts/`	`Roles.sol`, `WithdrawalLedger.sol`, `AuditTrail.sol`; `build/` holds ABI + BIN embedded in the binary
`cmd/sgchain/`	CLI: `keygen`, `deploy`, `bootstrap`, `serve`, `decrypt`
`internal/api/`	REST handlers, explorer page, OpenAPI + Swagger UI, team overview page
`internal/catalog/`	Regions and 10 demo banks (YAML)
`internal/sealing/`	Per-bank encryption (X25519 + HKDF + AES-GCM), commitments, Merkle root
`internal/ledger/`	Ledger interface; `fisco/` go-sdk adapter, `mock/` in-memory ledger, `ledgertest/` shared conformance suite
`internal/payout/`, `internal/keystore/`, `internal/hashing/`	Mock treasury, encryption keys, salted references and canonical JSON hashing

Path	Module
`configs/`, `scripts/`, `api.http`	Mock and FISCO configs; chain start/stop, demo, Solidity build; ready-made HTTP requests

3. Deployment

The lending platform deploys as one Docker container in about 3 minutes; the chain service is optional, because the platform can run with a built-in local ledger. `HANDOVER.md` in the repository is the full guide (HTTPS, backups, upgrades, troubleshooting).

Requirements

Item	Minimum	Recommended
OS	Any 64-bit Linux with Docker	Ubuntu Server 22.04 / 24.04 LTS
CPU / RAM	1 vCPU / 1 GB	2 vCPU / 2 GB
Disk	5 GB free	20 GB (logs, backups)
Software	Docker Engine 24+, Docker Compose v2	plus Nginx and Certbot for HTTPS
Without Docker	Python 3.12+	Node 18+ for the UI checks

No GPU is needed: the language model runs on a separate OpenAI-compatible (vLLM) host, and the system keeps working with template wording when it is off.

Option A: Docker (recommended)

```
git clone https://github.com/petertran1811/bouyup.git buoyup && cd buoyup
cp BackEnd/API_config/.env.example BackEnd/API_config/.env
nano BackEnd/API_config/.env          # fill the variables below
chmod 600 BackEnd/API_config/.env
docker compose up -d --build          # first build takes ~2-3 minutes
docker compose ps                     # STATUS must become "Up ... (healthy)"
```

Open `http://<host>:8000/`. The container runs as a non-root user (uid 10001); data lives in the `buoyup-data` volume, so rebuilding the container keeps it. Use `BUOYUP_PORT=8080 docker compose up -d` for another port.

Option B: without Docker

```
cp BackEnd/API_config/.env.example BackEnd/API_config/.env   # fill it
./run-local.sh               # creates BackEnd/.venv, installs, serves on
http://localhost:8000/
PORT=8080 ./run-local.sh     # another port
./run-local.sh test          # all test suites
./run-local.sh reset-lan     # reset the demo seller account
```

Configuration (`BackEnd/API_config/.env`)

Variable	Required	Meaning
`APP_ENV`	yes	`demo` (sample accounts for judging) or `production` (no sample accounts)
`CREDIT_LEDGER_KEY`	yes outside dev/test	HMAC key for the local ledger; generate with `python3 -c 'import secrets;print(secrets.token_hex(32))'`. The server refuses to start without it
`LLM_HOST`, `LLM_PORT`, `LLM_MODEL_NAME`, `LLM_API_KEY`	for LLM wording	OpenAI-compatible endpoint (vLLM)
`LLM_AGENT3_MODEL_NAME`	in production with LLM on	Must differ from `LLM_MODEL_NAME` (independent reviewer)
`BUOYUP_CHAIN_BACKEND`, `BUOYUP_CHAIN_URL`	yes	`remote` + `https://fs.buoyup.app` (live chain) or `local` (simulated ledger, no network)
`LLM_ENABLED`	no (`true`)	`false` runs every agent on its template path
`BUOYUP_AUTOPILOT`	no (`true`)	Background Agents 4b and 5
`BUOYUP_DEMO_PASSWORD`	no (`BuoyUp@2026`)	Password of the demo accounts
`SHOP_CONNECT_REQUIRED`	no (`true`)	Dossiers only from a shop whose ownership is proven

HTTPS

The browser allows the camera (face sign-in) only over HTTPS or localhost.

```
sudo apt-get install -y nginx certbot python3-certbot-nginx
sudo cp deploy/nginx-buoyup.conf /etc/nginx/sites-available/buoyup
sudo sed -i 's/buoyup.example.com/YOUR-DOMAIN/' /etc/nginx/sites-
available/buoyup
sudo ln -s /etc/nginx/sites-available/buoyup /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d YOUR-DOMAIN
```

Then bind the container to localhost only: in `docker-compose.yml` set `ports` to `"127.0.0.1:${BUOYUP_PORT:-8000}:8000"`.

Smoke test

```
curl -s $URL/healthz                                          #
{"ok":true,"version":"0.1.0"}
curl -s -o /dev/null -w '%{http_code}\n' $URL/app/giaodien/login.html    #
200
curl -s -X POST $URL/v4/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"admin@buoyup.vn","password":"BuoyUp@2026"}' | head -c 120   #
contains "token"
```

The start-up log should show `startup` with `llm_available: true`, `chain_catalog_synced` (10 banks) in remote mode, `autopilot_started` and `frontend_mounted`.

Chain service (`sgchain`)

In-memory mode, any OS, no chain needed:

```
go mod tidy
go test ./...
go run ./cmd/sgchain serve -config configs/mock.json     #
http://localhost:12080/docs/ and /explorer/
```

Real 4-node FISCO BCOS chain (Linux or WSL2):

```
CHAIN_DIR=~/sg-chain ./scripts/start_chain.sh            # 4 nodes, channel
ports 20200-20203
go run ./cmd/sgchain keygen -out keys                     # admin + operator
keys
# edit configs/fisco.json: set refSalt (>= 16 characters)
go run ./cmd/sgchain deploy -config configs/fisco.json   # deploy
WithdrawalLedger + AuditTrail, grant roles
go run ./cmd/sgchain serve  -config configs/fisco.json   #
http://localhost:12080
./scripts/demo.sh
```

For a real deployment set `apiToken` (BuoyUp sends it as `BUOYUP_CHAIN_TOKEN`) and `showDemoKeys: false`, then point BuoyUp's `BUOYUP_CHAIN_URL` at the service.

4. Test cases

The code ships with four test layers, all runnable offline: they use a fresh temporary database, turn the LLM off and force the local ledger, so they never write to the live chain. In the adversarial TestBench every case is an attack, and **PASS means the system resisted it**.

Suite	Location	Command (from repo root)	Result
Backend unit and integration	`BackEnd/src/tests` (45 files)	`cd BackEnd/src && ../.venv/bin/python -m pytest -q`	467 passing
TestBench security cases	`TestBench/cases/01-05`	`cd TestBench && ./run_testbench.sh`	31 / 31 resisted
TestBench 50 loan dossiers	`TestBench/cases/06_dossiers_bypass`	`cd TestBench && ../BackEnd/.venv/bin/python -m pytest -q cases/06_dossiers_bypass` then `make_dossier_sheet.py` (Excel report)	50 / 50 resisted (run of 17/10/2026)
API contract and UI checks	`Frontend/tests`	`BackEnd/.venv/bin/python -B Frontend/tests/v4-api-check.py`, then `cd Frontend/tests && node v4-ui-check.mjs`	9 API scenarios; UI groups
Post-loan services evidence	`BackEnd/src/scripts/demo_seller_services.py`	`./run-local.sh demo-evidence`	JSON + Markdown log in `TestBench/evidence/`
Chain service	Go packages	`go test ./...`; `make chain && make test-integration` for the live 4-node chain	Unit, API and conformance tests

`./run-local.sh test` runs the backend suite, the API contract check and the UI checks in one command.

Backend suite by area

Area	Test files (number of test functions)
Credit algorithm and data	`test_pipeline_algorithm` (13), `test_flow` (7), `test_fate_thin_seed` (9), `test_sources_fate` (10), `test_profiles` (8)
Agents and LLM controls	`test_llm_agents` (18), `test_agent2_fixes` (13), `test_agent3_fixes` (13), `test_arbitration` (12), `test_agent4_fixes` (12), `test_agent4_split` (8), `test_agent4_duties` (5), `test_agent5_fixes` (8), `test_agents45_llm` (7), `test_agent2_lang` (8), `test_gate_v4` (4), `test_ruleset_postaudit` (5), `test_llm_breaker` (2), `test_rag` (14)
End-to-end loan flow	`test_loan_circle` (21), `test_pipeline_flow` (11), `test_pipeline_scenarios` (8), `test_demo_samples` (19), `test_review_1710` (19)
Money and chain	`test_money_api` (18), `test_money_agent4` (12), `test_money_withdrawals` (9), `test_chain_sync` (12), `test_chain_remote` (8), `test_webank_inflows` (5), `test_autopilot` (7), `test_ledger_key` (2)
Identity and security	`test_shop_connect` (18), `test_face` (11), `test_consent` (8), `test_otp` (6), `test_auth` (4), `test_contract_routes` (2)
Post-loan services	`test_seller_services` (10), `test_seller_explain` (9), `test_seller_alerts` (8), `test_seller_reports` (6), `test_stats_daily` (6), `test_complaints` (9), `test_smaller_fixes` (5)

Parametrized tests expand these functions to the 469 collected cases. The remote-chain tests use a mock HTTP server (`httpx.MockTransport`), never the live chain.

TestBench security cases (31)

Category	Case	Attack resisted
Auth and session	AUTH-E1	Protected routes reject requests with no token (401)
Auth and session	AUTH-E2	Registration with a weak password is rejected (400)
Auth and session	AUTH-E3	Re-registering a used email is rejected (409), case-insensitively
Auth and session	AUTH-E4	The bank role cannot see seller applications or the officer queue (403)
Auth and session	AUTH-E5	Face re-check endpoints reject anonymous callers (401)
Auth and session	AUTH-M1	A seller token is refused on officer, admin and bank routes (403)
Auth and session	AUTH-M2	Role or bank code injected into the register body grants nothing
Auth and session	AUTH-M3	Repeated wrong passwords lock the account (429)
Auth and session	AUTH-M4	A token is rejected after logout
Auth and session	AUTH-M5	A seller cannot read, drive or list another seller's application (403)
Auth and session	AUTH-M6	Face-login frames with an undersized image or bogus session id are rejected
Auth and session	AUTH-H1	Forged, garbage or wrong-scheme bearer tokens do not authenticate
Auth and session	AUTH-H2	A flipped or truncated valid token is invalid (401)
Auth and session	AUTH-H3	Many wrong logins across accounts from one IP are rate-limited (429)
Auth and session	AUTH-H4	An enrolled face cannot be impersonated by a stranger, wrong account or photo
Auth and session	AUTH-H5	Repeated face-login starts from one IP are rate-limited (429)
Credit limit	CREDIT-M1	20-day inflow ignores money paid into an undeclared account

Category	Case	Attack resisted
Credit limit	CREDIT-M2	12-month average ignores money paid into an undeclared account
LLM, RAG, legal	LLM-E3	`verify_citation` rejects an invented article id and a non-verbatim quote
LLM, RAG, legal	LLM-M4	A verbatim quote from a law not yet in force is flagged
LLM, RAG, legal	LLM-H2	A compromised Agent 5 cannot change rates or gate, invent numbers or self-approve
LLM, RAG, legal	LLM-H3	A rule whose citation is not verbatim in the corpus is disabled at load
Multi-bank authz	API-E1	A seller cannot reach any `/admin` route
Multi-bank authz	API-E2	Sensitive GET routes reject a missing or forged token (401)
Multi-bank authz	API-E3	WeBank cannot read CMB's full catalogue entry; the master entry stays hidden
Multi-bank authz	API-E5	Out-of-range or unknown enum values are rejected (400)
Multi-bank authz	API-M1	The bank role is confined to bank routes
Multi-bank authz	API-M3	WeBank cannot edit CMB's policy or rotate CMB's public key
Multi-bank authz	API-M5	Injection-like strings in path or query never cause a 500 or bypass roles
Multi-bank authz	API-M6	Oversized, incomplete or non-JSON bodies are rejected; nothing is created
Multi-bank authz	API-H1	An officer is blocked from every admin-only route

TestBench 50 loan dossiers

Fifty sellers built from four reference dossiers (fashion, cosmetics, groceries, electronics) run the full flow through the API, including ID card OCR and the signed declaration. Difficulty runs from 1 to 5. The run of 17/10/2026 found three weaknesses (DS-07, DS-29, DS-40), which were fixed; the rerun resisted 50 of 50.

Case	Kind	Diff.	Technique	Expected secure outcome
DS-01	Honest	1	Standard household business	Approved; seed loan ⩽ 100,000
DS-02	Honest	1	Standard household business	Approved; type cap then seed ratio → ⩽ 100,000
DS-03	Honest	1	Personal seller	Approved as individual; ⩽ 25,000
DS-04	Honest	1	Standard household business	Approved; ⩽ 100,000
DS-05	Honest	1	Company	Approved; system tenor 24 months
DS-06	Honest	2	Statements backed by screenshots	Approved with the statement bonus
DS-07	Honest	2	Thin FATE, small seller	Approved under the thin-FATE cap; ⩽ 25,000
DS-08	Honest	1	Under 12 months of sales	Out of scope, eligibility date given
DS-09	Honest	1	Grade D	Not auto-approved; banks decline
DS-10	Honest	1	Personal seller	Approved; individual cap ⩽ 25,000
DS-11	False declaration	1	Wrong borrower type	Rejected: type does not match registration
DS-12	False declaration	1	Someone else's shop link	Not approved: owner does not match the ID card
DS-13	False declaration	1	E-cigarettes (prohibited online)	Rejected (LAW-GOODS)
DS-14	Flow abuse	1	Skip the declaration	API rejects the plan (400)
DS-15	Flow abuse	1	Call identity API directly	API rejects (400, no ID card)
DS-16	Flow abuse	1	Edit `tenor_m`	API rejects (tenor set by system)

Case	Kind	Diff.	Technique	Expected secure outcome
DS-17	Flow abuse	1	Edit `method`	API rejects (instalments only)
DS-18	Flow abuse	1	Edit the amount	API rejects (400)
DS-19	False declaration	2	Inflated purchase price	Limit unchanged (B3 uses invoice price)
DS-20	False declaration	2	Inflated capital need	Limit still capped
DS-21	False declaration	2	Hidden own capital	Limit still capped
DS-22	False declaration	2	Prohibited goods under another group	Rejected
DS-23	False declaration	2	Prohibited goods in a wrong group	Rejected
DS-24	False declaration	2	Licensed pharma goods in a wrong group	Not auto-approved
DS-25	False declaration	2	Licensed goods under another group	Not auto-approved
DS-26	Flow abuse	2	Loan stacking	Rejected: one loan per person
DS-27	Flow abuse	2	Amount above 50,000 for an individual	Limit ⩽ 25,000; larger amount rejected
DS-28	False declaration	3	Fake self-reported revenue	Not auto-approved (gap to tax > 30%)
DS-29	False declaration	3	Inflation within the 15% tolerance	No statement bonus
DS-30	False declaration	3	Unrelated screenshots	No bonus; note for the officer
DS-31	False declaration	3	Forged screenshots and figures	Not auto-approved (gap to tax)
DS-32	False declaration	3	Fake ID document	Officer review (RISK_ID_REVIEW)

Case	Kind	Diff.	Technique	Expected secure outcome
DS-33	Flow abuse	3	Dodge the expiry check	Officer review (back side missing)
DS-34	False declaration	3	Minor borrower	Rejected (LAW_ID_MINOR)
DS-35	False declaration	3	Expired ID card	Officer review
DS-36	False declaration	3	Borrowed ID card	Officer review (name differs from owner)
DS-37	False declaration	3	Dodge AML due diligence	Rejected: type mismatch
DS-38	Flow abuse	3	Prompt injection in chat	Proposal and limit unchanged
DS-39	False declaration	3	Prompt injection in data	Officer review (RISK-06)
DS-40	False declaration	4	Counterfeit goods without a brand field	Not auto-approved (IP)
DS-41	False declaration	4	Window-dressing before the loan	Not auto-approved (sales spike)
DS-42	False declaration	4	Cash-out through a related supplier	Not auto-approved (linked supplier)
DS-43	False declaration	4	Payee swap	Payout stopped, officer review
DS-44	Flow abuse	4	Double disbursement	Only one loan
DS-45	Flow abuse	4	Block post-loan data access	No signing, no payout
DS-46	Flow abuse	4	IDOR	Blocked (403/404)
DS-47	Flow abuse	4	Privilege escalation	Blocked (403)
DS-48	False declaration	5	Laundering through the WeBank account	Second loan not auto-approved (RISK_PASS_THROUGH)

Case	Kind	Diff.	Technique	Expected secure outcome
DS-49	False declaration	5	Misuse of funds	Red alert, recorded on-chain
DS-50	Flow abuse	5	Padding the dossier	Junk image rejected (422), duplicate rejected (409)

5. Demo accounts and walkthrough

With `APP_ENV=demo`, the server creates sample accounts on start-up; all use the password in `BUOYUP_DEMO_PASSWORD` (default `BuoyUp@2026`). Interactive API documentation is at `/docs`.

Role	Page	Account
Seller	`/app/giaodien/workspace.html`	`lan@buoyup.vn` (sample seller Ms. Lan)
Credit officer	`/app/giaodien/admin.html`	`officer@buoyup.vn`
Admin	`/app/giaodien/admin.html`	`admin@buoyup.vn`
Partner banks	`/app/giaodien/bank.html`	`webank@buoyup.vn`, `cmb@buoyup.vn`, `icbc@buoyup.vn`, `vcb@buoyup.vn`

A 10-minute walkthrough in the browser:

1. **Seller applies.** Sign in as `lan@buoyup.vn`. Choose household business, connect a sample shop (the code sent to the shop's registered phone is shown on screen in demo mode), scan the sample ID card from `Frontend/giaodien/mau-cccd/`, tick the three consents and sign by face or OTP, accept the suggested plan, and run verification. The result shows the limit approved by the banks and their shares.
2. **Seller signs.** Pick an amount within the limit and a due day, keep the cost table open for 20 seconds, enter the WeBank account and sign. The loan is disbursed; in remote chain mode the loan card shows the block number and 4/4 node confirmations.
3. **Loan life.** On the loan page, the "Simulation (demo)" panel adds sales inflows, spending and elapsed time, so debits, reminders, alerts and monthly reports appear.
4. **Officer review.** Sign in as `admin@buoyup.vn`, open the new dossier and the "Full flow" tab: blocks B1–B9 with formulas and real numbers, checked against the stored sheet and its on-chain seal. "Chain & ledger" shows the 4 nodes.

5. **Bank view.** Sign in as `webank@buoyup.vn`: the parallel-approval queue holds the case, and the bank decrypts its own share of the withdrawal.

Other sample shops, including the fictional Tmall shop "Nhã Cầm Gia Dụng", are listed by `GET /v4/public/demo-stores`. To start Ms. Lan over: `docker compose exec buoyup python /srv/BackEnd/src/scripts/reset_demo_seller.py` (or `./run-local.sh reset-lan`); old dossiers are archived, never deleted, because their on-chain records must stay.